

E-Commerce Sales Data

SQL-Based Product Performance Analysis Report

Engine: MySQL | Source: Ecommerce_sales_clean.csv

Prepared by

Shivam Vishwakarma

1. Executive Summary

This report documents the SQL-based product performance analysis carried out on the cleaned E-Commerce Sales dataset (2024–2025) inside a MySQL database. The cleaned CSV output from the earlier ETL process was loaded into a dedicated `ecommerce_sales` table, after which a series of SQL queries — aggregations, subqueries, CTEs (Common Table Expressions), and window functions — were used to analyze product-level sales, quantity, profit, profit margin, and discount performance.

The queries mirror and validate the findings from the Python/Pandas exploratory analysis, while also demonstrating SQL-specific techniques such as window functions (`RANK`, `DENSE_RANK`, `ROW_NUMBER`) for ranking products and correlated subqueries for identifying above-average performers.

2. Database & Table Setup

A new database, `ecommerce_sales`, was created, and a staging table `ecommerce_sales_data` was defined with 14 columns matching the cleaned CSV structure (Order ID, Order Date, Customer Name, Region, City, Category, Sub-Category, Product Name, Quantity, Unit Price, Discount, Sales, Profit, and Payment Mode).

```
CREATE TABLE ecommerce_sales_data (  
    Order_ID INT,  
    Order_Date DATE,  
    Customer_Name VARCHAR(100),  
    Region VARCHAR(50),  
    City VARCHAR(100),  
    Category VARCHAR(100),  
    Sub_Category VARCHAR(100),  
    Product_Name VARCHAR(255),  
    Quantity INT,  
    Unit_Price INT,  
    Discount INT,  
    Sales DECIMAL(12,2),  
    Profit DECIMAL(12,2),  
    Payment_Mode VARCHAR(50)  
);
```

The cleaned CSV file was loaded into the table using `LOAD DATA LOCAL INFILE`, and the table was subsequently renamed to `ecommerce_sales` for use in all analytical queries. A row-count check confirmed all 5,000 cleaned records loaded successfully.

```
LOAD DATA LOCAL INFILE "C:/Users/vis  
INTO TABLE ecommerce_sales_data  
FIELDS TERMINATED BY ','  
ENCLOSED BY ''''  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

```
ALTER TABLE ecommerce_sales_data  
RENAME TO ecommerce_sales ;
```

3. Product-wise Sales Analysis

The first analytical query aggregates total Sales by Product_Name across all orders to rank products by revenue contribution.

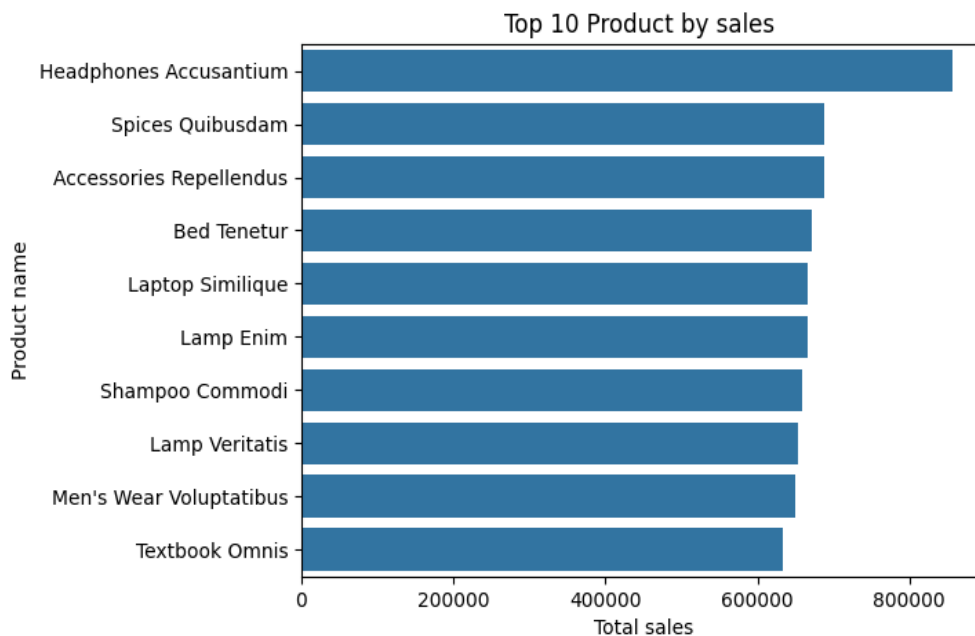
SELECT

```
Product_Name ,  
ROUND(sum(Sales),2) as Total_sales
```

FROM ecommerce_sales

GROUP BY Product_Name

ORDER BY Total_sales **DESC** **limit** 10 ;

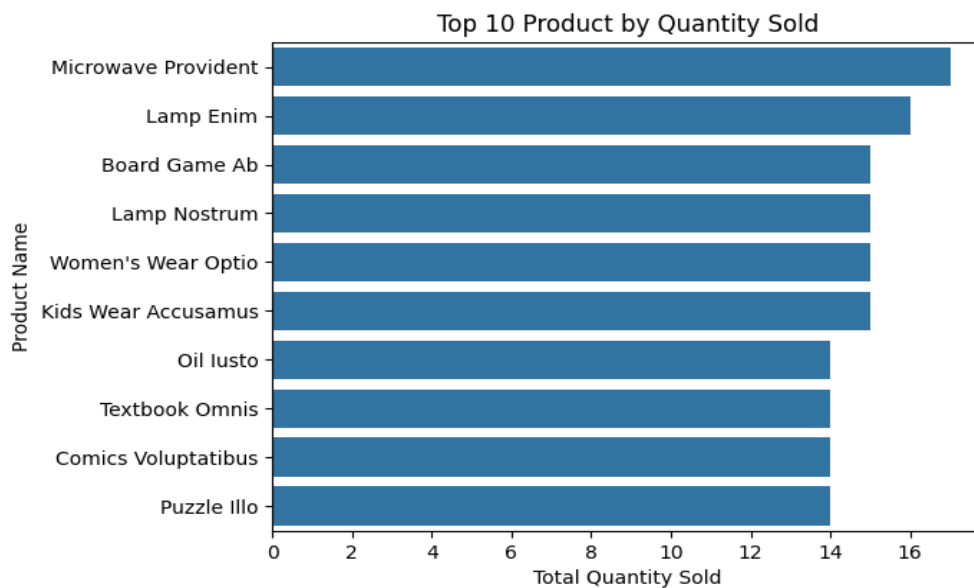


Rank	Product Name	Total Sales (₹)
1	Headphones Accusantium	8,57,184.20
2	Spices Quibusdam	6,87,651.25
3	Accessories Repellendus	6,87,083.30
4	Bed Tenetur	6,70,994.80
5	Laptop Similique	6,66,640.60
6	Lamp Enim	6,65,920.20
7	Shampoo Commodi	6,59,546.60
8	Lamp Veritatis	6,52,909.50
9	Men's Wear Voluptatibus	6,49,048.00
10	Textbook Omnis	6,33,246.25

4. Product-wise Quantity Analysis

Quantity sold was summed per product to identify high-volume products, useful for inventory and stock-replenishment decisions.

```
SELECT Product_Name,
SUM(Quantity) AS total_Quantity
FROM ecommerce_sales
GROUP BY Product_Name
ORDER BY total_Quantity DESC limit 10 ;
```

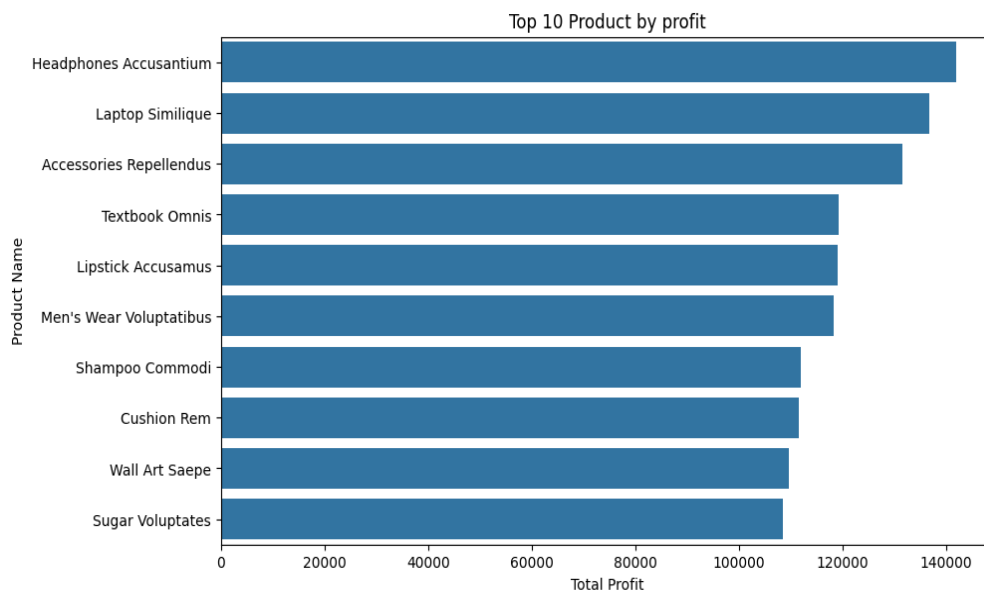


Rank	Product Name	Units Sold
1	Microwave Provident	17
2	Lamp Enim	16
3	Board Game Ab	15
4	Lamp Nostrum	15
5	Women's Wear Optio	15
6	Kids Wear Accusamus	15
7	Oil Iusto	14
8	Textbook Omnis	14
9	Comics Voluptatibus	14
10	Puzzle Illo	14

5. Product-wise Profit Analysis

Total profit was aggregated per product to identify which products contribute most to overall profitability, independent of raw sales volume.

```
SELECT Product_Name ,
ROUND(SUM(Profit),2) AS total_profit
FROM ecommerce_sales
GROUP BY Product_Name
ORDER BY total_profit DESC limit 10;
```

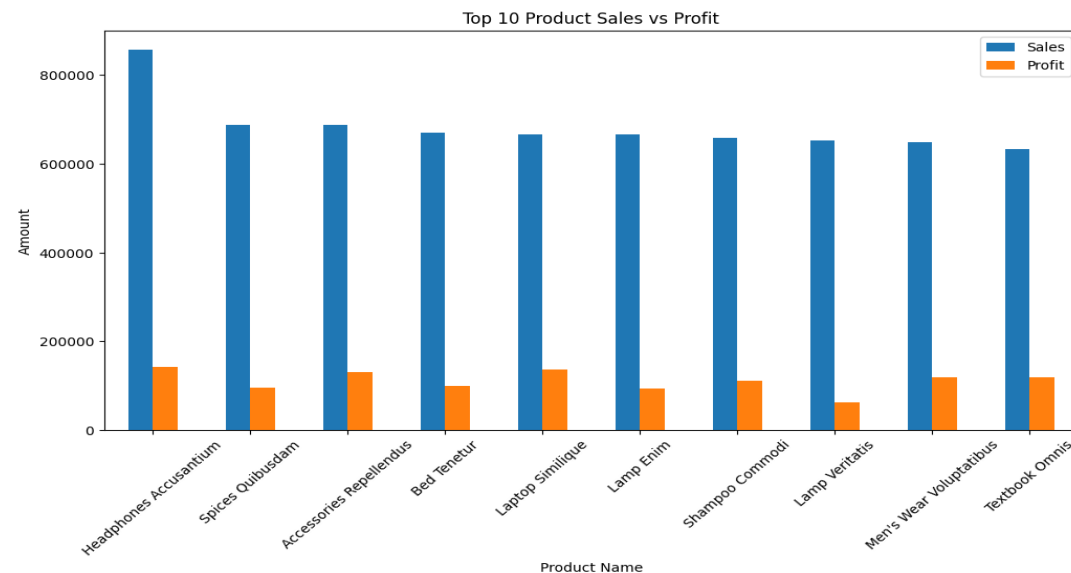


Rank	Product Name	Total Profit (₹)
1	Headphones Accusantium	1,41,873.79
2	Laptop Similique	1,36,751.80
3	Accessories Repellendus	1,31,526.41
4	Textbook Omnis	1,19,166.66
5	Lipstick Accusamus	1,19,017.65
6	Men's Wear Voluptatibus	1,18,300.59
7	Shampoo Comodi	1,11,950.64
8	Cushion Rem	1,11,530.63
9	Wall Art Saepe	1,09,548.98
10	Sugar Voluptates	1,08,494.16

6. Product-wise Sales vs. Profit

This query returns both Sales and Profit side-by-side for every product, letting the same result set answer both 'which products sell the most' and 'which products earn the most' at once.

```
SELECT Product_Name ,
ROUND(SUM(Sales),2) AS total_sales,
ROUND(SUM(Profit),2) AS total_profit
FROM ecommerce_sales
GROUP BY Product_Name
ORDER BY total_sales DESC limit 10 ;
```

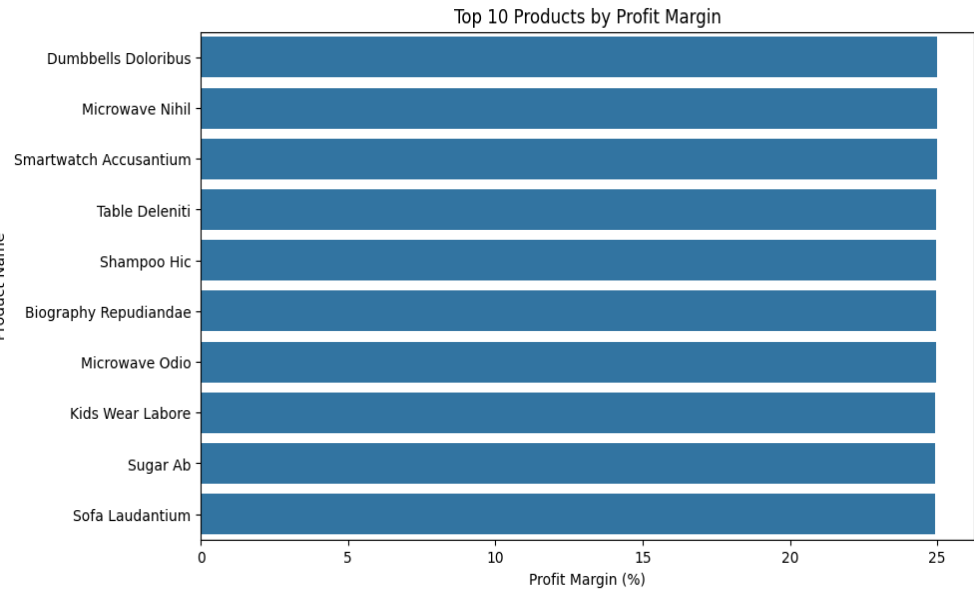


7. Product-wise Profit Margin

Profit Margin (%) is computed directly in SQL as $(\text{SUM}(\text{Profit}) / \text{SUM}(\text{Sales})) \times 100$, using NULLIF to guard against division by zero for any product with zero sales.

```
SELECT Product_Name ,
       ROUND(SUM(Sales),2) AS total_sales,
       ROUND(SUM(Profit),2) AS total_profit,
       ROUND(
         (SUM(Profit) / NULLIF (SUM(Sales),0)) *100 ,
         2
       ) AS profit_margin_percent

FROM ecommerce_sales
GROUP BY Product_Name
ORDER BY profit_margin_percent desc limit 10;
```

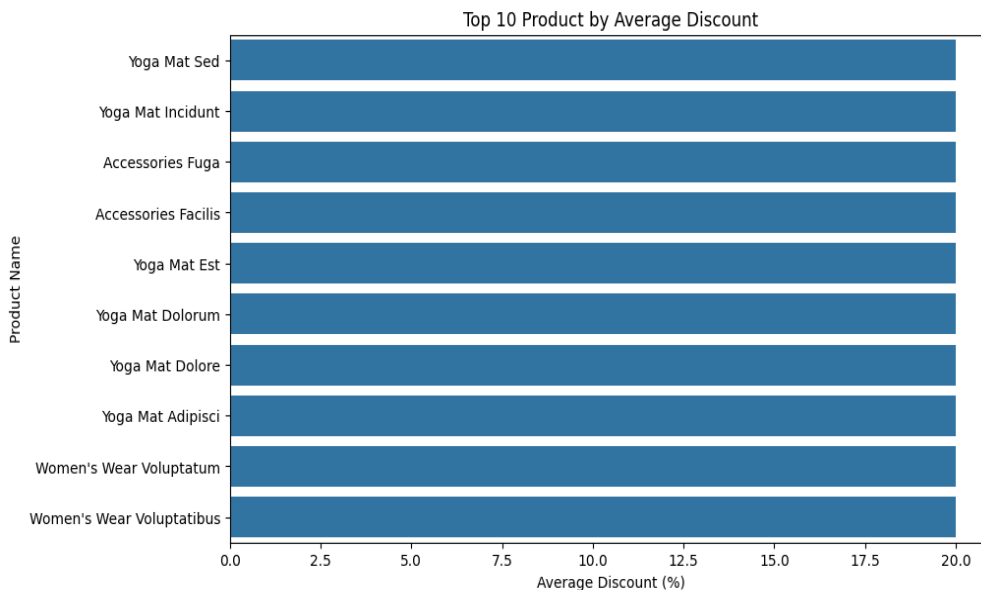


Rank	Product Name	Profit Margin (%)
1	Dumbbells Doloribus	24.996%
2	Microwave Nihil	24.979%
3	Smartwatch Accusantium	24.978%
4	Table Deleniti	24.974%
5	Shampoo Hic	24.959%
6	Biography Repudiandae	24.948%
7	Microwave Odio	24.947%
8	Kids Wear Labore	~24.9%

8. Product-wise Average Discount

Average discount percentage per product highlights which items are discounted most heavily — useful when reviewing promotional strategy against margin impact.

```
SELECT Product_Name ,  
ROUND(AVG(Discount),2) AS avg_discount  
FROM ecommerce_sales  
GROUP BY Product_Name  
order by avg_discount DESC limit 10;
```



The top discounted products — several Yoga Mat variants, Accessories, and Women's Wear items — each carry an average discount of 20%, the maximum discount level observed in the dataset.

9. Above-Average Performers

Two queries use a correlated subquery pattern to isolate products whose total Sales, or total Profit, exceed the average total Sales (or Profit) across all products — a way of separating consistently strong performers from the long tail.

```
SELECT Product_Name,
ROUND(SUM(Sales),2) AS total_sales
FROM ecommerce_sales
GROUP BY Product_Name
HAVING SUM(Sales) >(
    SELECT AVG(Product_Total_Sales)
FROM(
    SELECT
        Product_Name,
        SUM(Sales) AS Product_Total_Sales
    FROM ecommerce_sales
    GROUP BY Product_Name
) AS Product_Summary
)
ORDER BY total_Sales DESC limit 10 ;
```

The same pattern is repeated for Profit. Because the comparison threshold (the average of per-product totals) is computed from the full product list, products such as Headphones Accusantium, Spices Quibusdam, and Accessories Repellendus — already the leaders in the Top 10 by Sales and Profit tables above — comfortably clear this bar and appear at the top of both above-average lists.

10. Product Ranking with Window Functions

Three window-function variants were used to rank products by total sales, each handling ties differently:

- `RANK()` — leaves gaps in the ranking sequence after a tie (e.g., two products tied for 2nd are both ranked 2, and the next product is ranked 4).
- `DENSE_RANK()` — ranks tied products the same but does not skip the next rank number (the next product after a tie for 2nd is ranked 3).
- `ROW_NUMBER()` — assigns a strictly unique, sequential number to every row regardless of ties.

```

WITH Product_Summary AS (
    SELECT Product_Name, SUM(Sales) AS Total_Sales
    FROM ecommerce_sales
    GROUP BY Product_Name
),
Product_Ranking AS (
    SELECT Product_Name, Total_Sales,
           RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
    FROM Product_Summary
)
SELECT Product_Name, ROUND(Total_Sales,2) AS Total_Sales, Sales_Rank
FROM Product_Ranking
WHERE Sales_Rank <= 3
ORDER BY Sales_Rank;

```

Top 3 Products by Sales

Sales Rank	Product Name	Total Sales (₹)
1	Headphones Accusantium	8,57,184.20
2	Spices Quibusdam	6,87,651.25
3	Accessories Repellendus	6,87,083.30

11. Conclusion

The SQL analysis confirms and reinforces the findings from the earlier Python-based EDA: Headphones Accusantium remains the standout product, leading in both total sales and total profit, while Spices Quibusdam and Accessories Repellendus round out the top 3 by revenue.

- Aggregation queries (GROUP BY, SUM, AVG) provide the core product-level metrics — sales, quantity, profit, margin, and discount.
- Subqueries with HAVING isolate products performing above the field average, a useful filter for identifying consistently strong contributors.
- CTEs keep multi-step calculations (like profit margin) readable and reusable across queries.
- Window functions (RANK, DENSE_RANK, ROW_NUMBER) provide flexible ranking logic depending on how ties should be handled.

Together, these SQL queries validate the ETL and EDA work and provide a production-ready analytical layer that can be scheduled, automated, or plugged directly into BI tools such as Power BI for ongoing product performance monitoring.